

MINI PROJECT REPORT

GitHub Repository Code Explainer

A Local GenAI Application using Ollama, FastAPI and Streamlit

Student Name: HAMZAH TAREQ

Roll Number:2411021240077

Course / Department: BCA / GEN AI

Date: October 2026

Source code: <https://github.com/HAMZAHT-904/mini-project>

1. Introduction

When a developer or student opens an unfamiliar GitHub repository, it is often hard to understand what the project does, which technologies it uses, and which files matter. Reading every file takes a long time, especially for beginners and for large projects.

This mini project solves that problem. It is a local Generative AI (GenAI) application. The user pastes a GitHub repository link, and the application automatically reads the important source files and produces a simple-language explanation of the repository.

The explanation is generated by a Large Language Model (LLM) that runs locally on the user's own laptop through Ollama. No cloud AI service is used, so there are no API costs and the code never leaves the computer.

2. Objectives

- Accept a public GitHub repository URL from the user.
- Clone the repository and identify the relevant source-code files.
- Extract the code and prepare it for a language model without exceeding its limits.
- Send the code to a locally running LLM and receive a simple explanation.
- Display the explanation clearly in a web interface.
- Generate every explanation dynamically, with no hard-coded text.

3. Technologies Used

Component	Technology and purpose
Programming language	Python, used for the whole project.
Local LLM runtime	Ollama, which runs open-source models on the laptop and exposes a local REST API at http://localhost:11434 .
Language model	Qwen2.5 3B (about 2 GB). It is small enough for a normal student laptop and understands code well.
Backend	FastAPI for the web API, Pydantic for input validation, and Uvicorn as the server.
Repository processing	GitPython to clone repositories, plus standard Python file handling to read files as text.
Frontend	Streamlit, for the web page where the user enters the URL and reads the result.
HTTP communication	The requests library, used by the frontend and by the LLM service.

4. System Architecture

The application has three layers: a frontend, a backend, and a local LLM. The overall flow is:

GitHub URL > Streamlit frontend > FastAPI backend > GitPython clones the repository > file filtering and selection > code extraction > prompt sent to Ollama > LLM generates the explanation > FastAPI returns the result > Streamlit displays it.

The frontend never talks to the LLM directly. It only calls the backend endpoint POST /explain with the repository URL. The backend performs all the heavy work and returns a JSON object containing the explanation text.

4.1 Project structure

File	Purpose
frontend/app.py	Streamlit page: URL input, button, spinner, error messages, and display of the five result sections.
backend/main.py	FastAPI application with the POST /explain endpoint that runs the full pipeline.
backend/models.py	Pydantic models that validate the request and check the GitHub URL format.
backend/github_processor.py	Clones the repository, filters and ranks source files, applies size limits, and deletes the clone.
backend/llm_service.py	Builds the prompt and communicates with Ollama, including error handling.
backend/requirements.txt	List of Python libraries to install.
temp_repos/	Temporary folder where repositories are cloned, then deleted after each request.

5. How Each Component Works

5.1 Streamlit frontend

The frontend shows the title, a text box for the GitHub URL, and an Explain Repository button. When the button is pressed, it displays a loading spinner and sends the URL to the backend. When the answer arrives, it splits the text by headings and shows each section in its own expandable card. If the backend returns an error, the message is shown in a red error box.

5.2 FastAPI backend

FastAPI exposes one main endpoint, POST /explain. It receives {"repo_url": "..."}, then runs three steps in order: clone the repository, collect the code, and ask the LLM to explain it. A final cleanup step always deletes the cloned folder, even when an error occurs.

5.3 Pydantic validation

Pydantic checks that the request contains a `repo_url` and that it matches the pattern `https://github.com/username/repository`. Invalid input such as plain text is rejected immediately with a clear message, before any cloning is attempted.

5.4 GitPython repository cloning

GitPython clones the repository using a shallow clone (only the latest commit), which is much faster and smaller than a full clone. Password prompts are disabled, so private repositories fail with a clear message instead of hanging.

5.5 File filtering and selection

The processor walks through the cloned folder and keeps only useful files. It includes common source-code extensions (`.py`, `.js`, `.ts`, `.java`, `.cpp`, `.c`, `.h`, `.cs`, `.go`, `.rs`, `.php`, `.rb`, `.html`, `.css`, `.jsx`, `.tsx`, `.sql`) and important project files such as `README.md`, `package.json`, `requirements.txt` and `pom.xml`. It ignores folders such as `.git`, `node_modules`, `__pycache__`, `venv`, `dist` and `build`, and it skips binary files and files larger than 100 KB.

5.6 Ollama and the local LLM

The backend builds a prompt that contains the repository link, a list of files, and the selected code. It sends this prompt over HTTP to Ollama, which runs the Qwen2.5 3B model on the laptop. The model returns the explanation as text. The prompt instructs the model to use only the information provided and to write exactly five sections in simple language.

6. Token and Context Management

A small LLM can only read a limited amount of text at once, and repositories can contain thousands of files. Sending everything would be slow and would exceed the model's limit. The project therefore applies a simple priority-based limit:

1. **Priority score.** Each candidate file receives a score. `README.md` scores highest, then `package.json`, `requirements.txt` and `pom.xml`, then entry files such as `main.py`, `app.py`, `index.js` and `server.js`.
2. **Adjustments.** Files inside a `src` folder get a bonus, files closer to the project root rank higher, and test files are ranked lower.
3. **Limits.** Files are sorted by score, and the best ones are added until a limit is reached: at most 15 files, at most 3,000 characters per file, and at most 12,000 characters in total.
4. **File list.** A short list of up to 60 file names is also sent, so the model knows what else exists in the repository.
5. **Size check.** Repositories larger than 200 MB are rejected with a clear message.

This keeps the prompt within the model's context window while still giving it the most informative parts of the project.

7. Error Handling

Situation	Message shown to the user
Invalid GitHub URL	Invalid GitHub URL. Use the format <code>https://github.com/username/repository</code> .
Repository missing or private	Repository not found, or it is private. Only public repositories are supported.
Clone failure	Git clone failed. Check your internet connection and the URL.
No source files	No supported source-code files found.
Repository too large	Repository is too large (over 200 MB).
Ollama not running	Ollama is not running. Open the Ollama app or run <code>ollama serve</code> .
Model not installed	Model not installed. Run: <code>ollama pull qwen2.5:3b</code> .
Timeout	The model took too long to respond. Try a smaller model or repository.
Backend not started	Cannot reach the backend. Start it with <code>uvicorn</code> .

8. Security

- **No code execution.** The application never executes code from a cloned repository. Files are only opened and read as plain text.
- **Read-only use.** Nothing from the repository is imported, run, or evaluated, and no scripts or executables are started.
- **Safe filtering.** Binary files and very large files are skipped.
- **Temporary storage.** Each clone is stored in a randomly named folder and deleted after the request finishes.
- **Privacy.** All AI processing happens locally, so the code is not sent to any cloud service.

9. Testing and Results

The application was tested with the public repository <https://github.com/pallets/click>, which is the popular Python library Click for building command-line interfaces.

9.1 Test steps

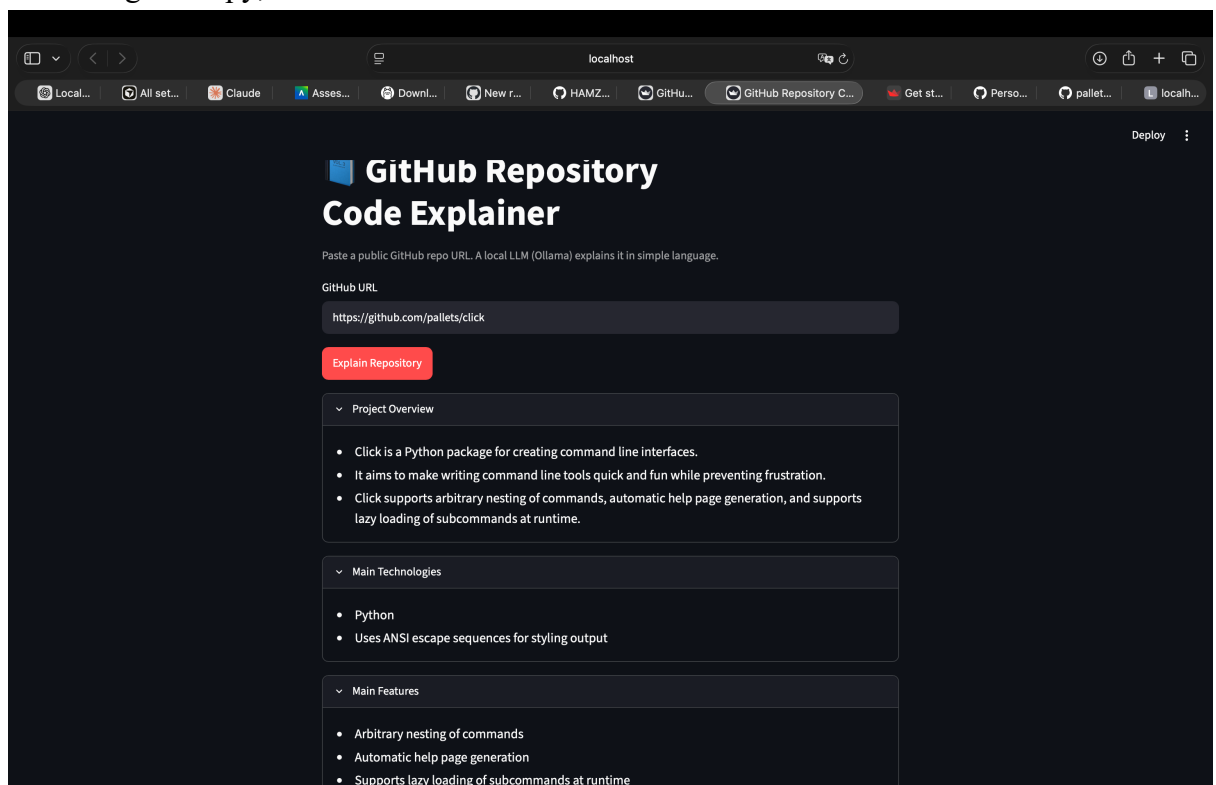
6. **Start Ollama.** Ollama was running with the `qwen2.5:3b` model installed.

7. **Start the backend.** The FastAPI backend was started with uvicorn on port 8000.
8. **Start the frontend.** The Streamlit frontend was started and opened at <http://localhost:8501>.
9. **Submit the URL.** The Click repository URL was pasted and the Explain Repository button was pressed.

9.2 Output obtained

The local LLM produced all five required sections:

- **Project Overview:** It described Click as a Python package for creating command-line interfaces that aims to make writing command-line tools quick and fun.
- **Main Technologies:** It identified Python as the main language and mentioned the use of ANSI escape sequences for styling output.
- **Main Features:** It listed arbitrary nesting of commands, automatic help page generation, lazy loading of subcommands at runtime, and styled output.
- **How It Works:** It explained that Click uses a decorator-based API to define commands and options, and generates help pages automatically.
- **Important Files:** It explained the purpose of README.md, src/click/__init__.py, src/click/globals.py, sr



c/click/_textwrap.py, src/click/_winconsole.py and example files.

This confirms that the full pipeline works: the repository was cloned, relevant files were selected, the local model generated the text, and the frontend displayed it in clear sections.

9.3 Observations

The explanation matches the actual purpose of the Click library. Because a small 3-billion-parameter model is used and only part of the code is sent, some details can be incomplete or emphasize minor files. This is expected and is discussed in the limitations section.

10. Why This Is a GenAI Application

The explanation is not copied from a template or looked up from a database. The LLM reads the repository's actual code and writes new natural-language text. A different repository produces a completely different explanation, and the same repository can be described in slightly different words each time. The project follows the pipeline GitHub Repository, Code Processing, Local LLM, Backend, Frontend, Explanation.

11. Limitations and Future Work

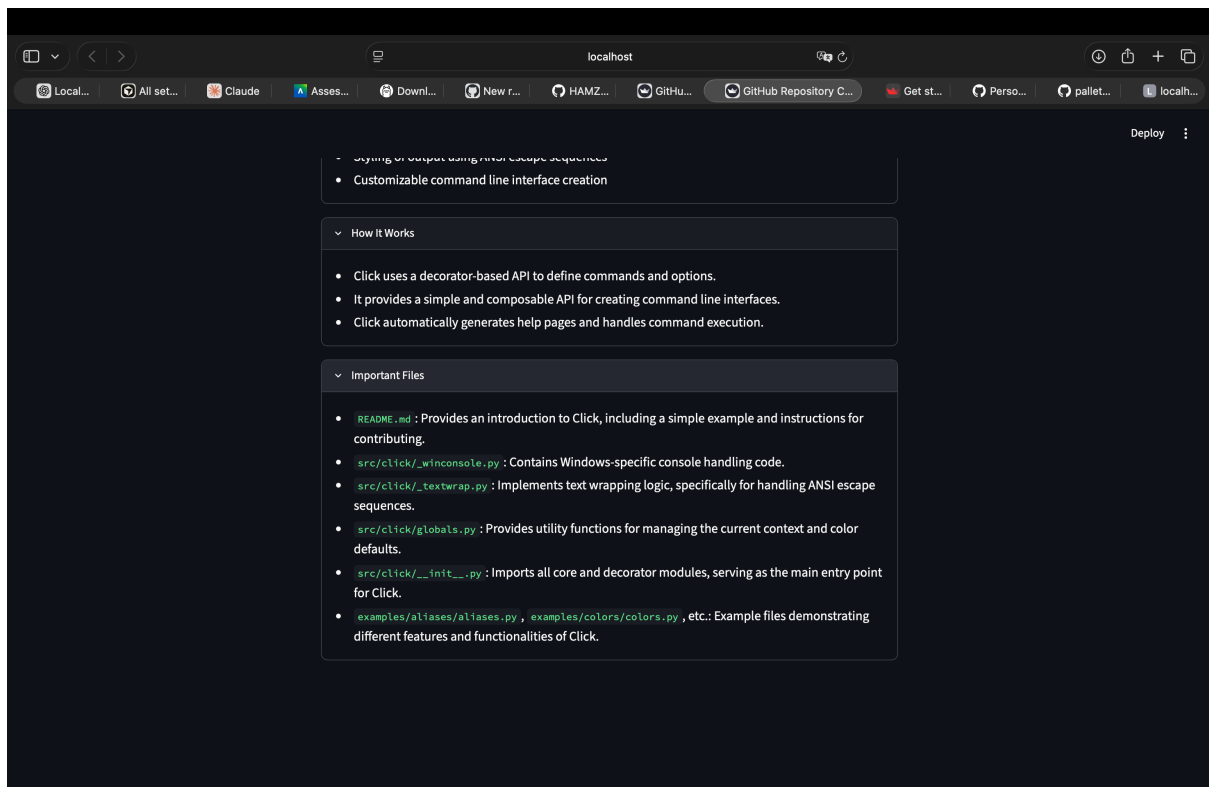
11.1 Limitations

- Only public GitHub repositories are supported.
- Only part of a large repository is analyzed because of the context limit, so details may be missed.
- A small local model can occasionally be less accurate or focus on minor files.
- Explanations take about one to three minutes on a CPU-only laptop.
- The application runs on the user's computer and cannot be hosted on free cloud platforms, because they cannot run Ollama.

11.2 Future improvements

- Show the answer gradually as the model writes it (streaming).
- Let the user choose a different model from a drop-down list.
- Summarize large repositories in stages, folder by folder.
- Add support for private repositories using an access token.
- Allow the explanation to be downloaded as a PDF or Markdown file.

12. Conclusion



The GitHub Repository Code Explainer demonstrates how a complete GenAI application is built from simple parts: GitPython for repository processing, FastAPI and Pydantic for the backend, Ollama for local LLM inference, and Streamlit for the user interface. The project meets its objectives: it accepts a GitHub URL, selects the most relevant code within safe size limits, never executes repository code, and produces a dynamic, simple-language explanation with a local open-source model.

13. How to Run the Project

10. **Prerequisites.** Install Git, Python and Ollama, then run: `ollama pull qwen2.5:3b`
11. **Python setup.** In the project folder run: `python3 -m venv venv`, then source `venv/bin/activate`, then `pip install -r backend/requirements.txt`
12. **Start the backend.** In the backend folder run: `uvicorn main:app --port 8000`
13. **Start the frontend.** In a second terminal, from the project root run: `streamlit run frontend/app.py`
14. **Use the app.** Open `http://localhost:8501`, paste a public GitHub URL, and click Explain Repository.